



UNIVERSIDADE DO MINHO
ESCOLA DE ENGENHARIA

RELATÓRIO

PROJETO DA U.C.
ANÁLISE E TESTE DE SOFTWARE

Grupo:

a91672 LUÍS FERREIRA
a93258 BERNARDO LIMA
a94523 ALEXANDRA SANTOS

Professores:

João Saraiva & José Nuno Macedo

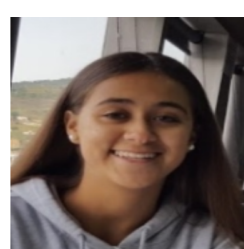
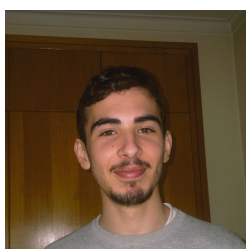




Tabela de conteúdos

1	Introdução	2
2	Ferramentas Utilizadas	3
2.1	JUnit	3
2.2	Evosuite	3
2.3	Jacoco	3
2.4	Pitest	3
2.5	Hypothesis	3
3	Trabalho TP1	4
3.1	Testes Unitários Manuais	4
3.1.1	Resultado do Jacoco - Testes quantitativos	5
3.1.2	Resultado do Pitest - Testes qualitativos	6
3.2	Evosuite	7
3.2.1	Resultado do Jacoco - Testes quantitativos	7
3.2.2	Resultado do Pitest - Testes qualitativos	9
4	Trabalho TP2	10
4.1	Testes Unitários Manuais	10
4.1.1	Resultado do Jacoco - Testes quantitativos	10
4.1.2	Resultado do Pitest - Testes qualitativos	10
4.2	Evosuite	11
4.2.1	Resultado do Jacoco - Testes quantitativos	12
4.2.2	Resultado do Pitest - Testes qualitativos	13
5	Hypothesis	15
5.1	TP3	15
5.1.1	Estratégia Abordada	15
6	Comparação entre TP1 e TP2	17
7	Conclusão	19



1 Introdução

Este trabalho foi desenvolvido no âmbito da Unidade Curricular de Análise e Teste de Software, com o objetivo de aplicar técnicas de teste ao projeto prático da disciplina de Programação Orientada a Objetos do ano letivo de 2022/2023, especificamente o projeto "Vintage". Serão analisadas as soluções fornecidas pelos docentes de POO.

As tarefas a serem realizadas incluem a escrita de testes unitários utilizando **JUnit**, complementando a suíte de testes existente se necessário. Será utilizado, também, o sistema **EvoSuite** para a geração automática de testes. A cobertura e a qualidade dos testes serão analisadas utilizando o sistema de mutação de código Java **PIT** para criar mutantes do programa para analisar a sua qualidade e o **jacoco** para analisar a sua cobertura. Além disso, será utilizado o **Hypothesis** para a geração automática de casos de teste, criando ficheiros de *logs* que a aplicação recebe como *input*.

Finalmente, será elaborado um relatório contendo gráficos e uma análise das soluções do projeto, seguindo as especificações do enunciado.

2 Ferramentas Utilizadas

2.1 JUnit

Os testes unitários são uma estratégia valiosa para garantir a precisão do código num programa. Geralmente, estes testes concentram-se em examinar individualmente métodos ou classes, permitindo validar o funcionamento do código através de uma variedade de casos de teste. O **JUnit** é utilizado como *framework* para testes unitários no caso da linguagem Java. Adicionalmente, utilizaremos a versão 4 da ferramenta de modo a maximizar a sua compatibilidade com todos os testes gerados.

2.2 EvoSuite

EvoSuite é uma ferramenta que gera automaticamente casos de teste para classes Java. Analisa o código-fonte e gera testes que visam cobrir diferentes caminhos de execução do programa. Esses testes gerados automaticamente podem economizar tempo e esforço para os desenvolvedores, fornecendo uma maneira rápida de iniciar a cobertura de testes num projeto.

2.3 Jacoco

JaCoCo, ou *Java Code Coverage*, é uma ferramenta utilizada para medir a cobertura do código em projetos Java. Analisa o código-fonte e fornece relatórios detalhados sobre quais partes do código foram executadas durante os testes. Com **JaCoCo**, os desenvolvedores podem identificar áreas do código que não estão a ser testadas adequadamente, permitindo uma melhoria na qualidade dos testes e, consequentemente, na qualidade do *software* final.

2.4 Pitest

Pitest é uma ferramenta de mutação de código para Java que ajuda os desenvolvedores a melhorar a eficácia dos seus testes automatizados. Introduce mutações (pequenas alterações) no código-fonte e executa os testes existentes para ver se as consegue detetar. Se os testes falharem ao detetar uma mutação, existe uma lacuna na cobertura de testes. **Pitest**, então, fornece informação valiosa sobre a qualidade dos testes, ajudando os desenvolvedores a escrever testes mais robustos e eficazes.

Para ser possível a avaliação usando o **Pitest** dos testes do **EvoSuite** foi necessário mudar em todos os ficheiros de testes gerados automaticamente a *flag* `separateClassLoader` para falso e também adicionar no **SureFire** a configuração `<testFailureIgnore>true</testFailureIgnore>` de modo a que sejam ignorados testes criados com falhas.

2.5 Hypothesis

Hypothesis é uma biblioteca Python que permite expressar propriedades sobre programas e testá-las com valores gerados aleatoriamente.

Um teste em **Hypothesis** consiste em duas partes: uma função semelhante a um teste normal na estrutura de teste à escolha, mas com alguns argumentos adicionais, e um decorador `@given` que especifica como fornecer esses argumentos. O decorador `@given` permite-nos especificar uma função que gera valores aleatórios para os argumentos do teste. O **Hypothesis** utiliza esses valores para executar o teste várias vezes com diferentes entradas, aumentando a confiança de que o teste está realmente a verificar a propriedade desejada.

3 Trabalho TP1

De todos os trabalhos, o TP1 foi aquele em que encontramos menos dificuldades na sua testagem, tendo sido bem desenvolvido e sendo o seu código simples e conciso de forma a ajudar na sua compreensão. Apesar disso este foi o único projeto que não apresentou a ferramenta de leitura de ficheiros *log* para execução automática de instruções, não podendo, deste modo, ser usado o **hypothesis**. Além disso, para facilitar a geração e avaliação de testes, refactorizamos o código do projeto de modo a eliminar *packages*, o que simplifica o uso do **EvoSuite** e do **Pitest**.

3.1 Testes Unitários Manuais

A escrita manual de testes **JUnit**, desempenha um papel crucial na verificação da qualidade e funcionalidade do projeto considerado. Ao contrário dos testes gerados automaticamente, os testes manuais são escritos com base no conhecimento do programador sobre o código. Este processo permite uma verificação mais concreta e personalizada das funcionalidades do sistema de *software*.

Neste contexto, foram criadas várias classes com o sufixo "*Test*" para desenvolver os testes unitários das principais classes do projeto "*Vintage*". Em cada uma destas classes, foram elaborados vários testes direcionados a métodos específicos ou, por vezes, a classes inteiras. Para validar os resultados obtidos em relação ao esperado, foram utilizadas funções do tipo *assert* que permitem realizar esse tipo de comparação. Para além de testes simples para cobrir uma grande base do código, também criamos um teste mais abrangente, **MegaTeste.java**, de modo a testar o maior número de funcionalidades possíveis simulando o decorrer normal do programa, tendo em conta a sua finalidade como uma loja *online* de roupa.

Teste da classe MegaTeste.java

```

1 @Test
2 public void complexTest() {
3     // User1 adiciona itens para venda
4     TShirt tshirt = new TShirt(TShirt.Tamanho.M, TShirt.Padiao.LISO, "T-Shirt", Artigo.
        Estado.NOVO, 0,
5         Artigo.Avalia o.IMPEC VEL, "T-Shirt", "Brand", "TS123", 20.0, 5.0,
            transportadora.getNome());
6     user1.adicionarPorVender(tshirt);
7
8     Sapatilhas sapatilhas = new Sapatilhas(42, true, "Red", LocalDate.of(2023, 1, 1),
9         Sapatilhas.Tipos_Sapatilhas.NORMAL, "Sapatilhas", Artigo.Estado.NOVO, 0,
10         Artigo.Avalia o.BOM,
            "Comfortable Shoes", "BrandB", "SP123", 50.0, 10.0, transportadora.getNome())
11     ;
12     user1.adicionarPorVender(sapatilhas);
13
14     // Verificar que os itens est o venda
15     assertEquals(2, user1.getPorVender().size());
16
17     // User2 cria uma encomenda
18     List<Artigo> carrinho = Arrays.asList(tshirt, sapatilhas);
19     Map<Integer, String> vendedores = new HashMap<>();
20     vendedores.put(tshirt.getId(), user1.getEmail());
21     vendedores.put(sapatilhas.getId(), user1.getEmail());
22
23     Encomenda encomenda = new Encomenda(user2.getEmail(), carrinho, vintage.
        calculaCustoExpedicao(carrinho),
24         LocalDate.now(), vendedores);
25     vintage.addEncomenda(encomenda);
26
27     // Verifica que a ordem foi adicionada
28     assertEquals(1, vintage.getEncomendas().size());
29
30     // P r sess o atual como user2
31     vintage.SetSessaoAtual(user2.getEmail());
32
33     // Processar a encomenda

```

```

33 vintage.trataEncomenda(Arrays.asList(tshirt.getId(), sapatilhas.getId()), new
    ArrayList<>(), new HashMap<>());
34
35 // Verificar que os itens foram removidos da lista de venda de User1
36 assertTrue(user1.getPorVender().isEmpty());
37
38 // Verificar que os itens foram adicionados lista de compras de User2
39 assertEquals(2, user2.getCompras().size());
40
41 // Finalizar a encomenda
42 encomenda.setEstado(Encomenda.Estado_Encomenda.FINALIZADA);
43 vintage.avancarTempo();
44 assertEquals(Encomenda.Estado_Encomenda.FINALIZADA, encomenda.getEstado());
45
46 // Verificar que o custo total da encomenda est correto
47 double totalCost = encomenda.getPrecoFinal();
48 double expectedCost = tshirt.precoFinal(LocalDate.now()) + sapatilhas.precoFinal(
    LocalDate.now())
49 + vintage.calculaCustoExpedicao(carrinho);
50 assertEquals(expectedCost, totalCost, 0.01);
51 }

```

3.1.1 Resultado do Jacoco - Testes quantitativos

A análise dos testes quantitativos do Trabalho Prático 1 com o **JaCoCo** revelam uma cobertura de código consideravelmente baixa nas classes testadas, cerca de 40%, devido a um baixo número de testes gerados. A criação de testes mais robustos foi o foco principal em vez de um maior número de testes, sendo esta a razão pela qual a cobertura de linhas foi pobre.

A estratégia a adotar para aprimorar a avaliação seria a adição de mais testes. Com a implementação desta melhoria, espera-se alcançar uma cobertura de testes mais abrangente e uma base de código mais sólida e confiável.

TP1											
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes			
Vintage		27%		21%	80 107	150 212	33 51	0 1			
Controlador		0%		0%	42 42	137 137	4 4	1 1			
ControladorArtigo		0%		0%	18 18	112 112	5 5	1 1			
ControladorEncomenda		0%		0%	29 29	86 86	10 10	1 1			
Output		0%		0%	22 22	61 61	10 10	1 1			
ViewMain		0%		n/a	12 12	40 40	12 12	1 1			
Input		0%		0%	16 16	55 55	7 7	1 1			
ControladorUtilizador		0%		0%	11 11	37 37	4 4	1 1			
Encomenda		69%		59%	18 51	20 118	2 29	0 1			
Utilizador		79%		39%	26 68	26 145	3 39	0 1			
ControladorTransportadoras		0%		0%	5 5	24 24	4 4	1 1			
Apresentacao		0%		n/a	17 17	37 37	17 17	1 1			
Malas		67%		54%	11 26	12 57	1 15	0 1			
Transportadoras		84%		64%	10 38	10 83	1 21	0 1			
Estados		0%		n/a	3 3	16 16	3 3	1 1			
TShirt		71%		68%	6 19	7 36	1 11	0 1			
ViewLogin		0%		n/a	2 2	5 5	2 2	1 1			
VintageAPP		0%		n/a	2 2	6 6	2 2	1 1			
Artigo		98%		57%	13 42	4 105	1 28	0 1			
Sapatilhas		98%		66%	8 29	1 67	0 17	0 1			
TShirt.Tamanho		100%		n/a	0 1	0 5	0 1	0 1			
Encomenda.Estado_Encomenda		100%		n/a	0 1	0 5	0 1	0 1			
TShirt.Padrao		100%		n/a	0 1	0 4	0 1	0 1			
Encomenda.Dimensao_Embalagem		100%		n/a	0 1	0 4	0 1	0 1			
Artigo.Avaliacao		100%		n/a	0 1	0 4	0 1	0 1			
Malas.Tipos_Malas		100%		n/a	0 1	0 3	0 1	0 1			
Sapatilhas.Tipos_Sapatilhas		100%		n/a	0 1	0 3	0 1	0 1			
Artigo.Estado		100%		n/a	0 1	0 3	0 1	0 1			
Total	3,963 of 6,708	40%	357 of 507	29%	351 567	846 1,470	122 299	12 28			

Figure 1: Resultados Jacoco TP1 - JUnit Test

3.1.2 Resultado do Pitest - Testes qualitativos

A análise dos resultados do **PitTest** destaca a eficácia dos testes gerados manualmente. Denota-se que apesar da sua baixa cobertura, devido à atenção dada e ao conhecimento do funcionamento do projeto, foi possível nos poucos testes criados uma grande eficácia na deteção de mutantes. Isto demonstra que apesar de o ser humano não ser capaz de realizar testes com tanta rapidez como por exemplo a ferramenta **EvoSuite**, é capaz de gerar melhor testes, sendo crucial a escrita manual de testes para complementar com testes gerados automaticamente.

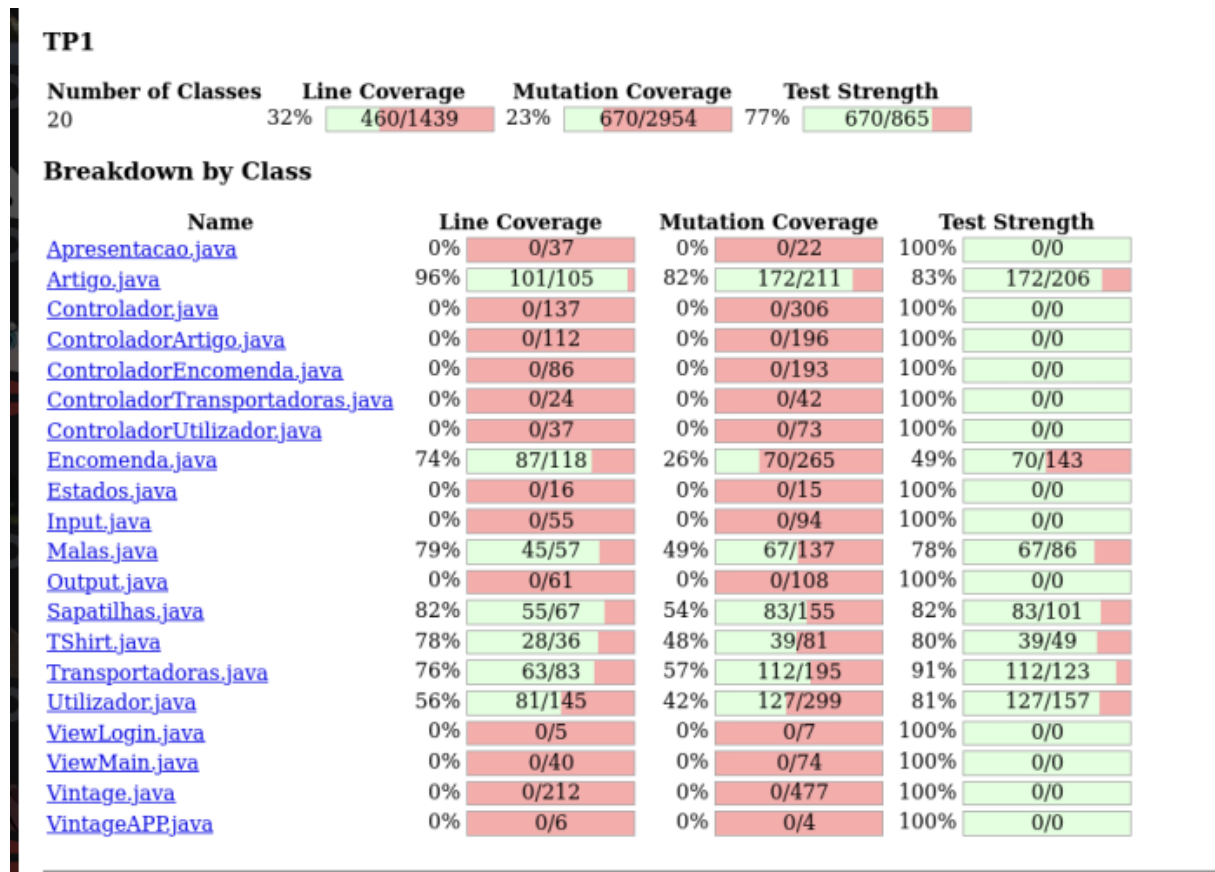


Figure 2: Resultados Pitest TP1 - JUnit Test

3.2 Evosuite

Durante a análise do TP1, exploramos também a ferramenta **Evosuite** como uma solução para a geração de testes automáticos. Esta ferramenta apesar de útil, não se revelou tão eficiente na cobertura dos métodos implementados quanto esperado, abrangendo perto de 70% .

Apesar de tudo, **Evosuite** demonstrou ser uma ferramenta que economiza tempo e esforço, ao automatizar a criação de testes. Para além disso, também proporcionou uma cobertura abrangente de testes simples, incluindo casos de erro e limites, o que nos concedeu um nível de confiança, na qualidade do código produzido.

Concluindo, **Evosuite** revelou ser uma ferramenta valiosa para a geração de testes durante o Trabalho Prático 1. Apesar dos desafios encontrados, a sua facilidade de uso, eficácia na geração de testes e abrangência de cenários de teste contribuíram significativamente para a melhoria da qualidade do código desenvolvido. Embora seja uma ferramenta promissora para práticas de teste de *software*, é crucial ressaltar a importância da personalização e revisão dos testes gerados para garantir a confiabilidade e qualidade dos resultados obtidos devido à simplicidade dos testes gerados.

3.2.1 Resultado do Jacoco - Testes quantitativos

A análise dos testes quantitativos do TP1, realizada com a ferramenta **Jacoco**, revela que a cobertura de código é de 73%, o que indica que 73% das linhas de código foram executadas durante os testes. A cobertura de ramificações (branches) é de 68%, o que significa que 68% das decisões condicionais foram testadas.

Ao analisar os resultados por elemento, observamos que os Controladores (*Controlador*, *ControladorArtigo*, *ControladorEncomenda* e *ControladorUtilizador*) possuem uma cobertura baixa, com valores entre 0% e 33% devido à sua baixa complexidade lógica e também a sua funcionalidade no projeto não necessitar de muitos testes.

Por outro lado, os restantes elementos apresentam uma cobertura alta, com valores entre 99% e 100%, o que indica que a maior parte do código desses elementos está a ser executada durante os testes, o que é um bom sinal de qualidade.

Em suma, os resultados dos testes quantitativos são positivos, com uma cobertura de código perto de 70%. Contudo, é importante lembrar que a cobertura de código não é o único indicador de qualidade dos testes, e que é necessário também avaliar a adequação dos casos de teste em relação aos requisitos do sistema.



TP1

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Controlador		8%		5%	39	42	121	137	2	4	0	1
ControladorArtigo		11%		4%	13	18	90	111	0	5	0	1
ControladorEncomenda		37%		42%	17	29	52	87	4	10	0	1
Vintage		81%		75%	25	107	32	234	7	51	0	1
ControladorUtilizador		21%		33%	6	11	24	37	1	4	0	1
Utilizador		88%		60%	13	68	10	153	1	39	0	1
ControladorTransportadoras		32%		100%	1	5	14	24	1	4	0	1
Input		85%		88%	1	16	8	56	0	7	0	1
VintageAPP		75%		n/a	0	2	2	6	0	2	0	1
Artigo		99%		89%	3	42	2	104	0	28	0	1
Output		99%		95%	1	22	0	61	0	10	0	1
Encomenda		100%		97%	1	51	0	122	0	29	0	1
Transportadoras		100%		100%	0	38	0	83	0	21	0	1
Sapatilhas		100%		100%	0	29	0	66	0	17	0	1
Malas		100%		100%	0	26	0	57	0	15	0	1
ViewMain		100%		n/a	0	12	0	40	0	12	0	1
TShirt		100%		100%	0	19	0	36	0	11	0	1
Apresentacao		100%		n/a	0	17	0	37	0	17	0	1
Estados		100%		n/a	0	3	0	17	0	3	0	1
TShirt.Tamanho		100%		n/a	0	1	0	5	0	1	0	1
Encomenda.Estado_Encomenda		100%		n/a	0	1	0	5	0	1	0	1
TShirt.Padiao		100%		n/a	0	1	0	4	0	1	0	1
Encomenda.Dimensao_Embalagem		100%		n/a	0	1	0	4	0	1	0	1
Artigo.Avaliação		100%		n/a	0	1	0	4	0	1	0	1
Malas.Tipos_Malas		100%		n/a	0	1	0	3	0	1	0	1
Sapatilhas.Tipos_Sapatilhas		100%		n/a	0	1	0	3	0	1	0	1
Artigo.Estado		100%		n/a	0	1	0	3	0	1	0	1
ViewLogin		100%		n/a	0	2	0	5	0	2	0	1
Total	1,757 of 6,721	73%	160 of 507	68%	120	567	355	1,504	16	299	0	28

Figure 3: Resultados Jacoco TP1 - Evosuite

3.2.2 Resultado do Pitest - Testes qualitativos

Em relação à cobertura de mutações, as classes *Transportadoras* e *ViewLogin* apresentam uma cobertura superior a 90%, indicando que os testes são eficazes em detectar erros no código dessas classes. No entanto, a maioria das classes tem uma cobertura de mutações inferior a 50%, sugerindo que os testes não são suficientes para detectar todos os possíveis erros no código. Particularmente, as classes *Estados*, *Input*, *Malas*, *Sapatilhas*, *TShirt*, *Utilizador* e *VintageAPP* apresentam uma cobertura de mutações de 0% ou muito baixa, o que significa que os testes atuais não conseguem detectar erros importantes nessas classes. Para resolver isso, é necessário aumentar a cobertura de mutações dessas classes, reformulando os testes para abranger diferentes tipos de erros e falhas no código. A utilização de técnicas como *Mutation Testing* pode ajudar a identificar mutações que os testes atuais não conseguem detectar.

No que abrange a força dos testes (*Test Strength*), algumas classes, como *Estados*, *Malas*, *Output* e *Sapatilhas*, apresentam um *Test Strength* de 100%, indicando que os testes cobrem os erros nos mutantes que interagem com esses testes. Todavia, a maioria das classes apresenta um *Test Strength* inferior a 75%, sugerindo que os testes não estão a cobrir a maioria dos cenários possíveis. Para melhorar a cobertura, é necessário desenvolver mais testes para aumentar o *Test Strength* das classes com baixo valor, especialmente aquelas com valores inferiores a 50%.

TP1

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
20	33% 483/1473	17% 520/2988	55% 520/946

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
Apresentacao.java	100% 37/37	50% 11/22	50% 11/22
Artigo.java	27% 28/104	11% 23/210	27% 23/86
Controlador.java	13% 18/137	3% 9/324	28% 9/32
ControladorArtigo.java	23% 25/111	9% 17/196	53% 17/32
ControladorEncomenda.java	40% 35/87	26% 52/199	63% 52/82
ControladorTransportadoras.java	54% 13/24	29% 12/42	57% 12/21
ControladorUtilizador.java	46% 17/37	19% 14/75	58% 14/24
Encomenda.java	25% 31/122	13% 36/279	34% 36/107
Estados.java	6% 1/17	0% 0/15	100% 0/0
Input.java	88% 49/56	69% 65/94	78% 65/83
Malas.java	0% 0/57	0% 0/136	100% 0/0
Output.java	100% 61/61	19% 21/108	19% 21/108
Sapatilhas.java	0% 0/66	0% 0/154	100% 0/0
TShirt.java	36% 13/36	9% 7/80	20% 7/35
Transportadoras.java	100% 83/83	92% 180/195	92% 180/195
Utilizador.java	1% 1/153	0% 0/298	100% 0/0
ViewLogin.java	100% 5/5	71% 5/7	71% 5/7
ViewMain.java	100% 40/40	55% 41/74	55% 41/74
Vintage.java	9% 21/234	5% 24/476	71% 24/34
VintageAPP.java	83% 5/6	75% 3/4	75% 3/4

Figure 4: Resultados Pitest TP1 - Evosuite

4 Trabalho TP2

4.1 Testes Unitários Manuais

Os testes unitários manuais criados para o TP2 tiveram como objetivo testar as funcionalidades do projeto de um modo específico e detalhado. Mais a baixo, podemos observar a análise dos resultados do **Jacoco** e do **Pitest**, e de que modo os testes unitários escritos manualmente completaram e melhoraram a cobertura de testes e a detecção de defeitos.

4.1.1 Resultado do Jacoco - Testes quantitativos

A análise dos testes quantitativos do Trabalho Prático 2 com o **JaCoCo** revelam uma cobertura de código ainda mais baixa do que nas classes do TP1, com 18% das linhas de código cobertas pelos testes. No entanto, a cobertura de ramos é significativamente menor, com apenas 10% de cobertura, o que significa que os testes não cobrem um bom número de linhas de código devido (novamente) a um baixo número de testes escritos pelo grupo.

Aspetos de melhoria seriam o aumento da cobertura dos testes, através da criação de mais testes unitários, que explorassem diferentes caminhos do código, incluindo cenários de erro, de modo a garantir que o código estivesse preparado para lidar com erros e exceções. Outra melhoria seria o aumento da variedade de testes, incluindo uma maior variedade de casos de teste.

TP2

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Menu		0%		0%	132	132	699	699	92	92	1	1
Vintage		0%		0%	122	122	378	378	93	93	1	1
Controlador		0%		0%	134	134	471	471	28	28	1	1
Gestor Artigos		0%		0%	122	122	191	191	44	44	1	1
Utilizador		23%		10%	81	106	188	251	46	68	0	1
Parsing		0%		0%	42	42	147	147	10	10	1	1
Gestor Utilizadores		0%		0%	61	61	106	106	46	46	1	1
Gestor Transportadoras		0%		0%	44	44	88	88	21	21	1	1
GestorEncomendas		50%		41%	20	36	29	71	6	19	0	1
Transportadora		66%		44%	38	61	47	132	13	36	0	1
Artigo		68%		26%	30	50	38	115	16	35	0	1
Fatura		72%		23%	22	40	26	90	9	27	0	1
Sapatilha		72%		28%	20	35	18	76	5	19	0	1
Encomenda		81%		63%	21	51	21	107	8	32	0	1
Mala		79%		42%	15	31	11	66	4	17	0	1
ComparadorVendas		0%		0%	4	4	6	6	2	2	1	1
Tshirt		87%		43%	7	19	2	34	0	11	0	1
Main		0%		n/a	2	2	6	6	2	2	1	1
EncomendaExistenteException		0%		n/a	1	1	2	2	1	1	1	1
ArtigoExistenteException		0%		n/a	1	1	2	2	1	1	1	1
TransportadoraNaoEncontradaException		0%		n/a	1	1	2	2	1	1	1	1
EncomendaDevolucaoImpossivelException		0%		n/a	1	1	2	2	1	1	1	1
CarrinhoVazioException		0%		n/a	1	1	2	2	1	1	1	1
ArtigoNaoEncontradoException		0%		n/a	1	1	2	2	1	1	1	1
EncomendaNaoEncontradaException		0%		n/a	1	1	2	2	1	1	1	1
Tamanho		100%		n/a	0	1	0	5	0	1	0	1
Padrao		100%		n/a	0	1	0	4	0	1	0	1
Dimensao		100%		n/a	0	1	0	4	0	1	0	1
Estado		100%		n/a	0	1	0	4	0	1	0	1
Total	10,839 of 13,349	18%	818 of 928	11%	924	1,103	2,486	3,065	452	613	16	29

Figure 5: Resultados Jacoco TP2 - JUnit

4.1.2 Resultado do Pitest - Testes qualitativos

A análise dos testes qualitativos do *Pitest* revelam que, apesar da cobertura de linhas, tal como no TP1, ter sido baixa, (o que indica que muito dos métodos da classe não estão a ser testados adequadamente) o *Test Strength* tem uma percentagem alta(74%). Isto

significa que grande parte dos testes estão a sobreviver aos mutantes, demonstrando a robustez dos mesmos.

Por exemplo, as classes *Menu*, *Parsing* e *GestorEncomendas*, possuem uma baixa cobertura de linhas e de mutações devido à pouca importância de serem testadas, enquanto que as classes *Controlador*, *Artigo* e *Vintage* já têm valores maiores dado que foram mais valorizadas na criação de testes, devido à sua importância no funcionamento do projeto.

Concluindo, o *Test Strength* é relativamente alto, mas é necessário melhorar a cobertura de linhas e a cobertura de mutação, de modo a garantir que todos os métodos e todas as mutações sejam testados.

TP2

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
17	10% 296/2843	7% 397/5400	74% 397/540

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
Artigo.java	35% 40/115	10% 24/229	45% 24/53
ComparatorVendas.java	0% 0/6	0% 0/19	100% 0/0
Controlador.java	0% 0/471	0% 0/642	100% 0/0
Encomenda.java	62% 66/107	33% 71/218	71% 71/100
Fatura.java	54% 49/90	49% 91/187	91% 91/100
GestorEncomendas.java	0% 0/71	0% 0/225	100% 0/0
Gestor Transportadoras.java	0% 0/88	0% 0/253	100% 0/0
Gestor Utilizadores.java	0% 0/106	0% 0/323	100% 0/0
Main.java	0% 0/6	0% 0/4	100% 0/0
Mala.java	52% 34/66	33% 51/155	82% 51/62
Menu.java	0% 0/699	0% 0/861	100% 0/0
Parsing.java	0% 0/147	0% 0/346	100% 0/0
Sapatilha.java	29% 22/76	11% 21/186	81% 21/26
Transportadora.java	64% 85/132	49% 139/285	70% 139/199
Tshirt.java	0% 0/34	0% 0/80	100% 0/0
Utilizador.java	0% 0/251	0% 0/577	100% 0/0
Vintage.java	0% 0/378	0% 0/810	100% 0/0

Figure 6: Resultados Pitest TP2 - JUnit

4.2 Evosuite

Durante a análise do Trabalho Prático 2 (TP2), exploramos novamente a ferramenta **Evosuite** para a geração automática de testes. Continuamos a utilizar o **Evosuite** e verificamos que a ferramenta ainda enfrenta desafios em alcançar uma boa cobertura em alguns módulos críticos.

Concluindo, apesar dos desafios encontrados, como a duração da avaliação (quatro horas e meia), a facilidade de uso e a eficácia na geração de testes contribuíram significativamente para melhorar a qualidade do código desenvolvido.

4.2.1 Resultado do Jacoco - Testes quantitativos

A análise dos resultados do **JaCoCo** revelam uma cobertura de código satisfatória em alguns componentes do sistema, mas com algumas áreas que necessitam de melhorias. Os componentes *Output*, *Transportadoras*, *ViewMain*, *Apresentacao* e *ViewLogin* apresentam cobertura de código de 100%, o que indica uma cobertura completa de testes nesses módulos. A maioria dos componentes do sistema alcançam uma cobertura acima de 90%, com exceção de *ControladorTransportadoras* e *Estados* que possuem uma cobertura menor.

No entanto, identificamos áreas que precisam de mais atenção: os módulos *ControladorTransportadoras* e *Estados* apresentam uma cobertura de código inferior a 10%, sinalizando uma lacuna significativa nos testes. É essencial que a equipa concentre os esforços adicionais nessas áreas de modo a garantir a qualidade do código e evitar eventuais falhas.

Observamos, também, uma inconsistência na cobertura de testes em diferentes áreas do sistema. Enquanto que alguns módulos, como *Input*, *ControladorUtilizador* e *TShirt*, têm uma cobertura de 0%, outros, como *Artigo* e *ControladorEncomenda*, apresentam uma cobertura mais elevada. Esta desigualdade na cobertura de testes precisa de ser investigada e corrigida para garantir uma abordagem uniforme e abrangente em todo o sistema.

Com base nesta análise, recomendamos priorizar a cobertura nos módulos *ControladorTransportadoras* e *Estados*, implementando testes adicionais nessas áreas críticas. Além disso, sugerimos uma investigação mais aprofundada para entender as razões por detrás da inconsistência na cobertura de testes e estabelecer diretrizes claras para garantir uma cobertura completa e consistente em todo o código.

Em suma, embora tenhamos alcançado progressos na garantia da qualidade do código através dos testes, é crucial direcionar esforços para abordar as áreas que requerem melhoria em relação à cobertura, para garantir a confiabilidade e estabilidade do sistema como um todo.

TP2

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Menu		17%		2%	107	132	620	740	67	92	0	1
Controlador		5%		1%	124	134	440	473	18	28	0	1
Vintage		25%		5%	108	122	315	389	79	93	0	1
Utilizador		34%		11%	75	106	150	253	37	68	0	1
Parsing		0%		0%	42	42	139	139	10	10	1	1
Gestor Utilizadores		66%		90%	21	61	33	112	19	46	0	1
Transportadora		59%		4%	40	61	53	132	15	36	0	1
Gestor Artigos		89%		83%	20	122	19	212	1	44	0	1
Sapatilha		78%		50%	20	35	18	74	6	19	0	1
Mala		78%		39%	17	31	13	66	5	17	0	1
Tshirt		78%		43%	9	19	7	34	2	11	0	1
Gestor Transportadoras		96%		93%	3	44	2	90	0	21	0	1
Encomenda		98%		94%	2	51	1	105	0	32	0	1
Artigo		99%		90%	3	50	2	115	0	35	0	1
GestorEncomendas		100%		100%	0	36	0	73	0	19	0	1
Fatura		100%		100%	0	40	0	90	0	27	0	1
Tamanho		100%		n/a	0	1	0	5	0	1	0	1
Padrao		100%		n/a	0	1	0	4	0	1	0	1
Dimensao		100%		n/a	0	1	0	4	0	1	0	1
Estado		100%		n/a	0	1	0	4	0	1	0	1
ComparadorVendas		100%		100%	0	4	0	6	0	2	0	1
Main		100%		n/a	0	2	0	6	0	2	0	1
EncomendaExistenteException		100%		n/a	0	1	0	2	0	1	0	1
ArtigoExistenteException		100%		n/a	0	1	0	2	0	1	0	1
TransportadoraNaoEncontradaException		100%		n/a	0	1	0	2	0	1	0	1
EncomendaDevolucaoImpossivelException		100%		n/a	0	1	0	2	0	1	0	1
CarrinhoVazioException		100%		n/a	0	1	0	2	0	1	0	1
ArtigoNaoEncontradoException		100%		n/a	0	1	0	2	0	1	0	1
EncomendaNaoEncontradaException		100%		n/a	0	1	0	2	0	1	0	1
Total	7,116 of 13,419	46%	549 of 928	40%	591	1,103	1,812	3,140	259	613	1	29

Figure 7: Resultados Jacoco TP2 - Evosuite

4.2.2 Resultado do Pitest - Testes qualitativos

O relatório de cobertura de testes gerado pelo **Pitest** demonstra que os testes gerados para o Trabalho TP2 possuem uma cobertura de linha de 46% e uma cobertura de mutação de 40%. Embora estes números não sejam os ideais, indicam que a aplicação possui um bom nível de cobertura de testes.

As classes **ComparadorVendas**, **Controlador**, **Mala** e **Tshirt** possuem 100% de cobertura de linha e de mutação, demonstrando uma cobertura de testes muito boa. As classes **Sapatilha** e **Transportadora** também possuem uma cobertura de linha a 100% e uma cobertura de mutação acima de 80%, o que indica que os testes são robustos. As classes **Parsing**, **Utilizador** e **Vintage** apresentam uma cobertura de linha acima de 70% e cobertura de mutação acima de 75%, indicando um nível de cobertura de testes aceitável.

Alguns aspetos a melhorar seriam as classes **GestorEncomendas**, **Fatura** e **Gestor Artigos**, que possuem uma cobertura de linha acima de 60% mas apresentam uma cobertura de mutação abaixo de 30%, o que sugere que os testes existentes podem não estar a cobrir adequadamente as diferentes linhas de código. Assim sendo, seria necessário rever os testes existentes e adicionar novos testes para cobrir os casos de mutação ausentes. Para as classes **Gestor Transportadoras** e **Gestor Utilizadores**, a cobertura de linha encontra-se acima de 50%, mas a cobertura de mutação fica em torno de 30%, indicando que os testes podem não estar a cobrir todos os casos de mutação. Na classe **Artigo** a cobertura de linha e de mutação está acima de 90%, porém ainda existem áreas que poderiam ser melhoradas. A classe **Encomenda** apresenta uma cobertura de linha acima de 70%, porém a cobertura de mutação é relativamente baixa (54%).

De um modo geral, o relatório do **Pitest** demonstra que os testes do Trabalho TP2 apresentam um nível de cobertura de linha relativamente bom. No entanto, a cobertura de mutações é baixa para algumas classes, indicando que os testes existentes não cobrem todos os casos de mutação.

Pit Test Coverage Report

Package Summary

TP2

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
18	46%	40%	75%

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
Artigo.java	98%	96%	100%
ComparatorVendas.java	100%	79%	79%
Controlador.java	0%	0%	100%
Encomenda.java	74%	54%	66%
Fatura.java	67%	13%	20%
GestorEncomendas.java	27%	2%	10%
Gestor_Artigos.java	62%	22%	41%
Gestor_Transportadoras.java	54%	12%	24%
Gestor_Utilizadores.java	51%	31%	64%
Main.java	0%	0%	100%
Mala.java	100%	91%	91%
Menu.java	0%	0%	0%
Parsing.java	36%	2%	15%
Sapatilha.java	100%	81%	81%
Transportadora.java	100%	100%	100%
Tshirt.java	100%	97%	97%
Utilizador.java	99%	76%	78%
Vintage.java	76%	83%	100%

Figure 8: Resultados PitTest TP2 - Evosuite

5 Hypothesis

Foi-nos proposta a implementação de um ficheiro semelhante ao ficheiro de *logs* de entrada do projeto de Programação Orientada a Objetos (POO). O grupo decidiu fazer essa implementação utilizando o projeto TP3. Para realizar esta tarefa, utilizou-se a biblioteca **Hyphotesis** e a linguagem Python.

5.1 TP3

Para o trabalho prático 3, foi necessário ter em conta as seguintes classes, onde é possível inserir via "log" objetos destas: *Utilizador*, *Transportadora*, *Malas*, *Malas Premium*, *Sapatilhas*, *Sapatilhas Premium* e *TShirt*.

A adição de comandos para estes tipos segue a seguinte arquitetura: `Data(dia/mes/ano), User(admin` proseguindo com a ação pretendida. Antes de ser possível escolher tal ação, foi necessário restringir certos tipos de dados:

- User:email - Para que fosse possível gerar *emails* considerados válidos pelo programa, o endereço de email precisava conter um número qualquer de caracteres, seguido de um '@', outro número de caracteres, um ponto ('.') e mais alguns caracteres.
- User:nif - Terá de ser um valor do tipo *integer*;
- Artigo:Stock - Qualquer tipo de artigo (Mala, Mala Premium, TShirt, Sapatilha ou Sapatilha Premium), ao ser adicionado via "*script*" deverá estar entre os valores 1 e 20.
- Artigo:Ano - Terá de ser um *integer* entre 1000 e 3000;
- Artigo:Depreciation - Terá de ser um *integer* entre 0 e 100;
- TShirt:Size - Terá de ser um entre quatro tipos: "small", "medium", "large" e "extra large";
- TShirt:Pattern - Terá de corresponder a um de 3 padrões, "plain", "stripes" e "palm trees";

Os restantes dados ou são do tipo *integer*, gerados através do *st.integers()*, utilizando os parâmetros *min_value* e *max_value* ou do tipo *string*, através do *st.fromRegex*, de modo a que o texto gerado não incluísse caracteres especiais.

5.1.1 Estratégia Abordada

Para ser possível a inserção dos dados gerados corretamente, seguiu-se uma abordagem com foco numa estrutura global com os seguintes tipos:

- emails: Correspondem aos *id*'s dos utilizadores gerados;
- transportadoras: Correspondem às *transportadoras* não *premium* geradas;
- transportadorasP: Correspondem às *transportadoras premium* geradas;
- itemStock: Corresponde a um *dictionary* onde são armazenados todos os artigos gerados;
- userOrders: Corresponde às ordens por efetuar por um dado utilizador gerado;

- `userOrdered`: Corresponde às ordens efetuadas por um dado utilizador gerado;
- `userOrdersId`: Corresponde ao *Id* de um utilizador que efetuou certa *order*;

Em relação ao uso de **geradores**, inicialmente foram criados geradores incapazes de seleccionar apenas um índice válido entre as estruturas, o que nos impediu de gerar valores válidos para serem introduzidos posteriormente. Para contornar essa falha, a ideia foi criar cinco geradores diferentes, cada um correspondente a um dos cinco tipos de ações distintas que um utilizador poderia realizar:

- **create user**, onde dado um email, username, nome, endereço e um nif, o comando geraria um utilizador;
- **carrier create**, onde dado um nome geraria um carrier, podendo este ser do tipo **Premium** ou não;
- **item create [Artigo]**, podendo criar um artigo dos tipos referidos anteriormente, sendo que cada artigo recebe parametros ligeiramente diferentes;
- **item stock [add/remove]**, podendo adicionar ou remover stock a artigos existentes;
- **cart [add/order/return]**, podendo adicionar ao carrinho de compras, efetuar uma encomenda e pedir a anulação da mesma, sendo que esta anulação implica que a data inserida no início do comando tenha uma data posterior em 2 dias, do agendamento da mesma;

Para a geração do texto propriamente dito, foram utilizados geradores de inteiros (`st.integers()`) para que fosse criado um número diferente de utilizadores, transportadoras, artigos, mudanças no stock e por fim, mudanças no carrinho de compras.

6 Comparação entre TP1 e TP2

Durante a análise dos trabalhos práticos TP1 e TP2 observamos diferenças significativas em termos de dificuldades encontradas, correções do código e desenvolvimento geral dos projetos.

No caso do TP1, enfrentamos menos dificuldades na fase de testagem. O código foi desenvolvido de uma maneira simples e concisa, o que facilitou a sua compreensão e manipulação. No entanto, o projeto apresentou duas limitações notáveis, a ausência de uma ferramenta de leitura de ficheiros *log* para a execução automática de instruções (o que impediu o uso da ferramenta Hypothesis) e o uso de *packages* que complicou o uso do **Pitest** tendo que ter sido necessário refatorizar o projeto de modo a que não existissem.

No que diz respeito aos testes unitários manuais no TP1, foram criadas várias classes de teste para verificar as funcionalidades das principais classes do projeto "Vintage". Utilizamos asserções para comparar os resultados obtidos com os esperados e criamos um teste mais abrangente, denominado *MegaTeste*, que simulou o funcionamento normal do programa como uma loja online de roupas. No entanto, a análise quantitativa dos testes com o *JaCoCo* revelou uma cobertura de código relativamente baixa (40%), devido ao foco na criação de testes robustos em vez de uma maior quantidade de testes. A análise qualitativa com o *Pitest* destacou a eficácia dos testes gerados manualmente, que, apesar da baixa cobertura, foram eficazes na detecção de mutantes, o que demonstra a importância dos testes manuais.

Para além do *JaCoCo*, exploramos também o *EvoSuite* para o TP1, que apesar de proporcionar uma cobertura de aproximadamente 70%, gerou alguns testes inválidos, o que levou à implementação de uma alternativa com o intuito de circunventar esse problema, através das configurações no plugin **SureFire**. Esta abordagem, embora não tenha alcançado uma cobertura ideal em todos os métodos, foi valiosa na economia de tempo e esforço, além de oferecer uma boa abrangência de cenários de teste. Comparativamente, a análise dos resultados do *JaCoCo* mostrou uma cobertura de código de 73% e de ramificações de 68%, com controladores apresentando uma cobertura baixa, enquanto outros elementos tiveram cobertura alta (entre 99% e 100%), face à análise do *Pitest* que revelou uma cobertura de mutações variada, com algumas classes alcançando menos de 50% e outras acima de 90%. O *Test Strength* variou em algumas classes, tendo atingido 100%, mas a maioria ficou abaixo dos 75%.

No TP2 enfrentamos mais dificuldades, como a escrita de métodos com caracteres especiais (ç, ã) ou cálculos mal definidos, o que se refletiu numa menor cobertura de código e ramificações. A cobertura quantitativa com o *JaCoCo* foi de apenas 18% nas linhas de código e 10% nas ramificações, destacando a necessidade de aumentar a quantidade e a variedade de testes para melhorar a cobertura. Com a deteção dos defeitos feita e posterior correção do projeto, foi possível proporcionar uma melhor análise do mesmo. A análise qualitativa com o *Pitest* mostrou que, embora a cobertura de linhas fosse baixa, o *Test Strength* foi relativamente alto (74%), o que indicou a robustez dos testes existentes. Contudo, houve inconsistências na qualidade dos componentes, com alguns alcançando 100% de cobertura e outros significativamente baixos.

A utilização do *EvoSuite* no TP2 continuou a enfrentar desafios na cobertura dos módulos críticos. A análise do *JaCoCo* revelou uma cobertura satisfatória em alguns com-

ponentes, mas com algumas áreas necessitadas de melhorias, como os módulos *ControladorTransportadoras* e *Estados*, que apresentaram uma cobertura inferior a 10%. Houve uma inconsistência na cobertura dos testes em diferentes áreas do sistema, com alguns módulos apresentando 0% de cobertura enquanto outros apresentaram valores elevados. Quanto ao **Pitest**, os seus valores apresentaram-se aceitáveis apesar da grande demora na avaliação dos testes. Presume-se que isto deverá estar relacionado com o mau estado do código, criando dificuldades à ferramenta de avaliação de testes devido a alguns destes necessitarem de leitura de inputs para serem executados.

Em suma, o TP1 foi mais fácil de testar e encontrava-se melhor desenvolvido, apesar da limitação de não existir a leitura de um ficheiro de *log*. O TP2, por sua vez, apresentou maiores desafios devido à sua pior implementação. Ambos necessitaram das suas mudanças pontuais, tendo sido necessária em ambos a mudança manual nos testes criados automaticamente de modo a que pudessem ser avaliados (editando a flag do **EvoSuite** `separateClassLoader` para falso). Apesar disso, ambos os projetos demonstraram um bom entendimento do paradigma da programação orientada aos objetos, mas com níveis diferentes de qualidade.

7 Conclusão

A realização deste trabalho permitiu aplicar e aprofundar os conhecimentos adquiridos na disciplina de Análise e Teste de Software, utilizando várias ferramentas e abordagens para assegurar a qualidade e robustez do código do projeto "Vintage".

A implementação de testes unitários manuais com *JUnit* possibilitou uma verificação precisa e detalhada das funcionalidades do projeto, destacando-se pela sua capacidade de detetar defeitos específicos e proporcionar uma validação mais personalizada. Os testes gerados manualmente mostraram-se eficazes na deteção de mutações, mesmo com uma cobertura de código relativamente baixa.

A utilização da ferramenta *Evosuite* para geração automática de testes demonstrou ser uma abordagem eficiente para aumentar a cobertura de código rapidamente, cobrindo diversos cenários de teste de forma automática. No entanto, a simplicidade dos testes gerados requer uma revisão e personalização para garantir a sua eficácia.

As análises de cobertura realizadas com *JaCoCo* revelaram áreas do código que necessitam de melhorias, tanto em termos de cobertura de linhas como de ramos. A inclusão de mais testes unitários, explorando diferentes caminhos do código, é essencial para aumentar a confiabilidade do *software*.

A utilização do *Pitest* permitiu avaliar a qualidade dos testes através da técnica de mutação de código, destacando a importância de criar testes robustos que consigam detetar mutações e, consequentemente, potenciais falhas no código.

Por fim, a exploração da biblioteca *Hypothesis* para geração automática de casos de teste com valores aleatórios mostrou-se uma ferramenta promissora para aumentar a cobertura de testes e garantir a qualidade do *software* de forma contínua.

Em suma, a combinação de diferentes ferramentas e abordagens de teste possibilitou uma análise abrangente e detalhada dos projetos "Vintage", tendo sido concluído que a melhor abordagem para a análise de projetos não é só uma das técnicas abordadas neste projeto, mas sim a junção de todas.